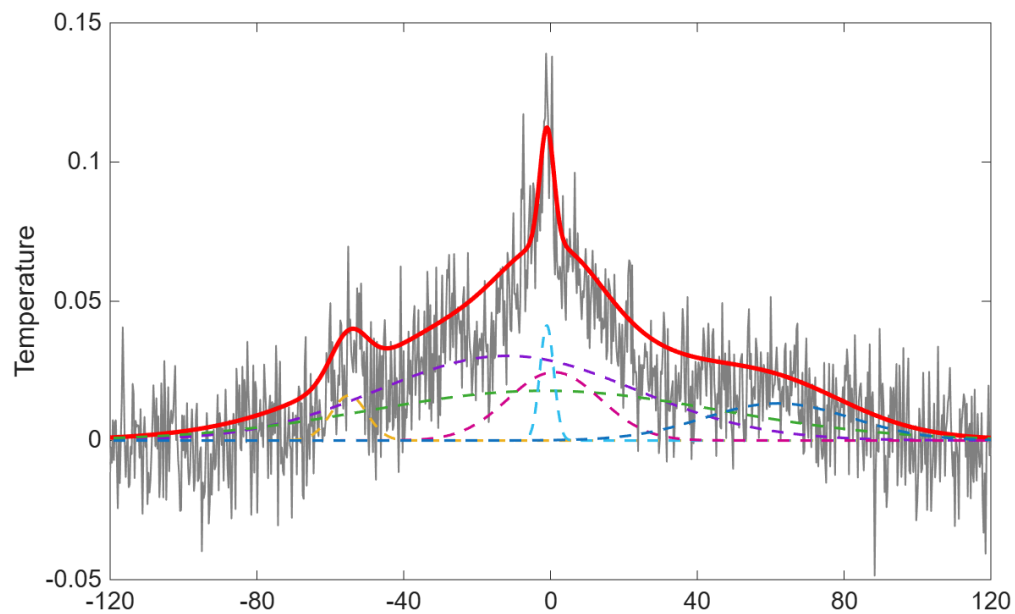


Inferencia Bayesiana Paralela con MCMC

Ajuste de mezclas gaussianas en señales experimentales



High Performance Computing - HPCF

Autor: Luis Eduardo Cardoso Paez

Profesor: Edwin Flórez Gómez

Universidad de Puerto Rico

10 de junio de 2026

Resumen

En este proyecto hacemos uso de la Inferencia Bayesiana para la estimación de parámetros en señales astronómicas. Construimos un modelo de suma de picos gaussianos, que tenga como parámetro de entrada el número posibles picos, k , que pueda tener la señal. Para hacer esto, implementamos un algoritmo de Markov Chain Monte Carlo (MCMC) con el método de Metropoli-Hastings para estimar los parámetros de todos los k picos gaussianos superpuestos con el ruido. Desde el enfoque de HPC usamos el paralelismo a nivel de tareas ejecutando 4 cadenas MCMC independientes simultáneamente en diferentes núcleos de la computadora o de clúster, esto permite una exploración eficiente del espacio de parámetros, mostrando si un pico detectado aparece en todas las cadenas. Finalmente, los resultados de todas las cadenas se combinan mediante un promedio para obtener una estimación general de los parámetros de cada pico.

1. Descripción del Proyecto

El proyecto estima, a partir de un archivo CSV con una señal experimental, los parámetros de cada k picos gaussianos. Estos parámetros son:

- A — Amplitud de cada pico el cual esta relacionado con la intensidad del gas
- μ — Centro (posición) de cada pico que relacionada con la velocidad a la que se mueve
- σ — Ancho (desviación estándar) de cada pico que relacionado con la temperatura del gas

El modelo asume la forma:

$$y(x) = \sum_{i=1}^k A_i \cdot \exp\left(-\frac{(x - \mu_i)^2}{2\sigma_i^2}\right) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_{\text{ruido}}^2) \quad (1)$$

Donde k , es la cantidad de picos que se cree que tiene la señal astronómica.

1.1. Fundamento Bayesiano

La inferencia bayesiana se basa en el teorema de Bayes:

$$P(\theta|\text{Datos}) = \frac{P(\text{Datos}|\theta) \cdot P(\theta)}{P(\text{Datos})} \quad (2)$$

Aplicando logaritmo:

$$\ln P(\theta|\text{Datos}) = \ln P(\text{Datos}|\theta) + \ln P(\theta) - \ln P(\text{Datos}) \quad (3)$$

La ecuación (3), es la que usaremos para hacer la inferencia basados en el modelo que se muestra a continuación.

1.1.1. Verosimilitud (Likelihood) - $P(\text{Datos}|\theta)$

La verosimilitud mide la capacidad del modelo para explicar los datos observados:

$$\ln P(\text{Datos}|\theta) = -\frac{1}{2} \sum_i (y_i - \hat{y}_i)^2 \sigma^2 \quad (4)$$

Donde, y_i son los datos observados, $\hat{y}_i$ es el valor predicho por el modelo, σ^2 es la varianza del ruido.

1.1.2. Distribución a Priori - $P(\theta)$

La distribución a priori incorpora restricciones físicas y conocimiento previo del sistema:

$$\begin{aligned} A &> 0 \quad (\text{amplitud positiva}) \\ \sigma &> 0 \quad (\text{ancho positivo}) \\ \mu &\in [x_{\min}, x_{\max}] \quad (\text{centro dentro del rango de medición}) \end{aligned} \quad (5)$$

1.1.3. Distribución a Posteriori - $P(\theta|\text{Datos})$

La distribución a posteriori representa la probabilidad actualizada de los parámetros después de observar los datos:

$$P(\theta|\text{Datos}) \propto \mathcal{L} \cdot \text{Prior} \quad (6)$$

Es decir:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior} \quad (7)$$

Esta distribución combina la información de los datos observados (verosimilitud) con el conocimiento previo (prior) para obtener una estimación más robusta de los parámetros.

1.2. Tipo de Paralelismo

Para este proyecto usaremos **Task Parallelism** (Paralelismo a nivel de tareas):

- Se ejecutan 4 cadenas MCMC independientes simultáneamente, cada una en un núcleo distinto del clúster o de la maquina local.
- Cada cadena usa una semilla aleatoria diferente ($\text{seed} = 1000 + \text{chain_id}$), garantizando exploración diversa del espacio de parámetros.
- Al finalizar, los resultados de cada cadena se combinan promediando.

Core 1: Cadena 1 (seed=1001)	→ result_chain_1.txt
Core 2: Cadena 2 (seed=1002)	→ result_chain_2.txt
Core 3: Cadena 3 (seed=1003)	→ result_chain_3.txt
Core 4: Cadena 4 (seed=1004)	→ result_chain_4.txt

Figura 1: Esquema de paralelización de cadenas MCMC

Ventajas sobre el modo lineal

- El tiempo de ejecución es similar al de 1 sola cadena
- Mejor exploración del espacio de parámetros
- Verificación de posibles picos para evitar falsos positivos.

2. Archivos del Repositorio

Aunque se tienen señales astronómicas reales para poder realizar el experimento NO TENGO PERMITIDO COMPARTIR estas señales hasta que las investigaciones que se están haciendo con esas hayan concluido. Por tal motivo, podemos crear señales sintéticas para hacer el experimento. En el repositorio vamos a encontrar los siguientes archivo:

Listing 1: Estructura del repositorio

```

1  generar_senal.py          # Genera la senal sintetica con K picos
   gaussianos
2  bayes_parallel.py        # Algoritmo MCMC paralelo (una cadena por
   ejecucion)
3  analisis_cadenas.py      # Combina resultados y genera graficas
4  README.md               # Este archivo

```

Donde la rutina generar_senal.py una senal sintética con k picos gaussianos. En este caso, hay que ajustar los parámetros para cada picos en el programa, cuando lo ejecutamos nos genera un archivo llamado senal.csv que va ser el parámetros de entrada de bayes_parallel.py. En bayes_parallel.py se tiene todo el modelo Bayesiano que se encarga de hacer la estimación de los parámetros, tiene como parámetros de entrada la señal, el número de picos k y la cantidad de iteraciones del método MCMC. En analisis_cadenas.py combinamos los resultados y generamos las gráficas que se muestran en pantalla y previamente se guardan en la carpeta. EL procedimiento para ejecutarlo se muestra a continuación.

3. Cómo Ejecutarlo

Para poder ejecutar este programa, tenemos que tener instalado pandas. Si no está instalado, lo podemos hacer como:

```

1  pip install numpy pandas matplotlib scipy

```

3.1. Generar la señal sintética (opcional)

```
1 python generar_senal.py
```

Esto crea `senal.csv` con 3 picos gaussianos y ruido gaussiano ($\sigma = 0,02$).

3.2. Ejecutar las cadenas MCMC en paralelo

En el clúster (Linux):

```
1 for id in 1 2 3 4; do
2     python3 bayes_parallel.py --file senal.csv --K 3 --chain_id $id
3     --n_iter 15000 &
4 done
5 wait
6 echo "todas las cadenas finalizadas"
```

En Windows (PowerShell / VS Code):

Los parametros de entrada son la `senal.csv`, el número de picos $k = 3$ y el número de interacciones $n_{iter} = 20000$.

```
1 for %i in (1 2 3 4) do start python bayes_parallel.py --file senal.
  csv --K 3 --chain_id %i --n_iter 20000
```

Esto genera los archivos:

- `result_chain_1.txt`
- `result_chain_2.txt`
- `result_chain_3.txt`
- `result_chain_4.txt`

3.3. Analizar resultados y generar gráficas

```
1 python analisis_cadenas.py
```

Produce:

- Tabla en consola con los parámetros estimados vs valores reales y % de error
- `analisis_cadenas.png` — Comparación de las 4 cadenas por parámetro
- `senal_reconstruida.png` — Señal reconstruida vs señal original

3.4. Descargar resultados del clúster (PSCP)

```
1 pscp lcardoso@boqueron.hpcf.upr.edu:/home/lcardoso/*.png C:\Users\
  profesor\Desktop\
2 pscp lcardoso@boqueron.hpcf.upr.edu:/home/lcardoso/*.txt C:\Users\
  profesor\Desktop\
```

4. Parámetros del Algoritmo

Parámetro	Valor	Descripción
-file	senal.csv	Archivo CSV con columnas x, y
-K	3	Número de picos gaussianos
-chain_id	1-4	ID de la cadena (define la semilla)
-n_iter	20000	Número de iteraciones MCMC
sigma_noise	0.02	Nivel de ruido de la señal
burn_in	30 %	Fracción descartada al inicio

Cuadro 1: Parámetros de configuración del algoritmo

5. Resultados Obtenidos

Parámetro	Valor Real	Estimado	Error
A_1	0.3	~ 0.2639	$\sim 12\%$
A_2	0.2	~ 0.1950	$\sim 2.5\%$
A_3	0.1	~ 0.0872	$\sim 12\%$
μ_1	-80	~ -80.0006	$\sim 0.00075\%$
μ_2	-5	~ -5.1402	$\sim 2.8\%$
μ_3	100	~ 99.8047	$\sim 0.19\%$
σ_1	2	~ 2.6271	$\sim 31\%$
σ_2	4	~ 4.2280	$\sim 5\%$
σ_3	3	~ 3.8433	$\sim 25\%$

Cuadro 2: Resultados de estimación de parámetros

6. Código Fuente

6.1. generar_senal.py

Listing 2: Código para generar señal sintética

```

1 import math
2 import random
3 import csv
4
5
6 n_puntos = 10000
7 x_min = -200
8 x_max = 200
9 paso = (x_max - x_min) / (n_puntos - 1)
10
11
12 picos = [
13     {"A": 0.2, "mu": -5, "sigma": 4},
14     {"A": 0.3, "mu": -80, "sigma": 2},
15     {"A": 0.1, "mu": 100, "sigma": 3}
16 ]
17
18
19 nivel_ruido = 0.02
20 random.seed(42)
21 nombre_archivo = "senal.csv"
22
23 with open(nombre_archivo, mode='w', newline='') as archivo_csv:
24     escritor = csv.writer(archivo_csv)
25
26     escritor.writerow(["X", "Senal_Limpia", "Senal_Con_Ruido"])
27
28     for i in range(n_puntos):
29         x = x_min + i * paso
30
31
32         senal_limpia = 0.0
33         for pico in picos:
34             exponente = -((x - pico["mu"]) ** 2) / (2 * (pico["sigma"] ** 2))
35             senal_limpia += pico["A"] * math.exp(exponente)
36
37
38         ruido = random.gauss(0, nivel_ruido)
39         senal_con_ruido = senal_limpia + ruido
40
41         escritor.writerow([round(x, 4), round(senal_limpia, 4),
42                             round(senal_con_ruido, 4)])
43
44 print(f" Listo ! Archivo '{nombre_archivo}' generado con xito .")

```

```

44 print("Contiene 1000 puntos con picos en X=[2.5, 5.0, 7.5] y ruido
    moderado.")

```

6.2. bayes_parallel.py

Listing 3: Algoritmo MCMC paralelo

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import find_peaks
4 import argparse
5 import pandas as pd
6 import sys
7
8 parser = argparse.ArgumentParser()
9 parser.add_argument("--file", type=str, required=True)
10 parser.add_argument("--K", type=int, required=True)
11 parser.add_argument("--chain_id", type=int, default=0)
12 parser.add_argument("--n_iter", type=int, default=15000)
13
14 args = parser.parse_args()
15 # Creamos una semilla diferente para cada cadena. porque daria el
16 # mismo resultado.
17 np.random.seed(1000 + args.chain_id)
18
19 K = args.K
20 data = pd.read_csv(args.file)
21
22 try:
23     col_x = [c for c in data.columns if c.lower() in ['x', '
24         tiempo_o_x', 'time', 'fre']][0]
25     col_y = [c for c in data.columns if c.lower() in ['y', '
26         senal_con_ruido', 'temperature', 'amplitude']][0]
27
28     x = data[col_x].values
29     y = data[col_y].values
30     print(f"[Cadena {args.chain_id}] Columnas detectadas con xito
31         -> X: '{col_x}', Y: '{col_y}'")
32 except IndexError:
33     print(f" ERROR: No se pudieron mapear las columnas en '{args.
34         file}'.")
35     print(f"Columnas disponibles en tu archivo: {list(data.columns)}
36         ")
37     sys.exit(1)
38
39 # =====

```



```

35 # Modelos de suma de gaussianas
36 # =====
37 def gaussian_mixture(x, A, mu, sigma):
38     y_out = np.zeros_like(x)
39     for Ai, mi, si in zip(A, mu, sigma):
40         y_out += Ai * np.exp(-(x - mi)**2 / (2 * si**2))
41     return y_out
42 # LIKELIHOOD
43 def log_likelihood(theta, x, y, K, sigma_noise):
44     A = theta[0:K]
45     mu = theta[K:2*K]
46     sigma = theta[2*K:3*K]
47     y_hat = gaussian_mixture(x, A, mu, sigma)
48     return -0.5 * np.sum((y - y_hat)**2 / sigma_noise**2)
49
50 # PRIOR
51 def log_prior(theta, K, xmin, xmax):
52     A = theta[0:K]
53     mu = theta[K:2*K]
54     sigma = theta[2*K:3*K]
55
56     if np.any(sigma < 0.01):
57         return -np.inf
58     if np.any(mu < xmin) or np.any(mu > xmax):
59         return -np.inf
60     if np.any(A < 0):
61         return -np.inf
62     lp = 0
63     lp -= np.sum((A/20)**2)
64     lp -= np.sum((sigma/10)**2)
65     return lp
66
67 # POSTERIOR
68 def log_posterior(theta, x, y, K, sigma_noise):
69     xmin, xmax = np.min(x), np.max(x)
70     lp = log_prior(theta, K, xmin, xmax)
71     if not np.isfinite(lp):
72         return -np.inf
73     return lp + log_likelihood(theta, x, y, K, sigma_noise)
74
75
76 # ORDENAMIENTO
77
78
79 def sort_parameters(theta, K):
80     A = theta[0:K]
81     mu = theta[K:2*K]

```

```

82     sigma = theta[2*K:3*K]
83     idx    = np.argsort(mu)
84     return np.concatenate([A[idx], mu[idx], sigma[idx]])
85
86
87 def initialize(x, y, K):
88
89     #peaks, props = find_peaks(y, distance=50, height=0.05)
90     peaks, props = find_peaks(y, distance=max(1, len(x)//(20*K)),
91                               height=np.max(y)*0.02)
92
93     if len(peaks) >= K:
94         top_idx = np.argsort(props["peak_heights"])[::-1][:K]
95         peaks    = peaks[top_idx]
96     else:
97
98         peaks = np.linspace(0, len(x)-1, K).astype(int)
99
100     A      = y[peaks]
101     mu     = x[peaks]
102
103     sigma = np.ones(K) * (x.max() - x.min()) / 100
104
105     return np.concatenate([A, mu, sigma])
106
107
108
109
110 # =====
111 # MCMC
112 # =====
113
114 def run_mcmc(x, y, K, n_iter=20000):
115     sigma_noise = 0.02 #DEJAMOS EL RUIDO CONSNTANTE PARA ESTE
116                        #EXPERIMENTO
117     theta_init = initialize(x, y, K)
118     perturbacion = np.random.normal(0, 0.05, size=len(theta_init))
119     theta = theta_init + perturbacion
120     theta = sort_parameters(theta, K)
121
122     current_lp = log_posterior(theta, x, y, K, sigma_noise)
123
124     samples = []
125     accept  = 0
126

```

```

127     rango          = x.max() - x.min()
128     step_A         = 0.05
129     step_mu        = rango * 0.001
130     step_sigma     = 0.05
131
132     steps = np.concatenate([
133         np.ones(K) * step_A,
134         np.ones(K) * step_mu,
135         np.ones(K) * step_sigma
136     ])
137
138     for i in range(n_iter):
139
140         proposal     = theta + np.random.normal(0, 1, size=len(theta))
141                     * steps
142         proposal     = sort_parameters(proposal, K)
143         prop_lp      = log_posterior(proposal, x, y, K, sigma_noise)
144
145         if np.log(np.random.rand()) < (prop_lp - current_lp):
146             theta    = proposal
147             current_lp = prop_lp
148             accept    += 1
149
150         samples.append(theta.copy())
151
152         if i % 1000 == 0 and i > 0:
153             print(f"[Cadena {args.chain_id}] Iter {i}/{n_iter}, Tasa
154                 de aceptaci n parcial: {accept/(i+1):.3f}")
155
156     print(f"          [Cadena {args.chain_id}] Finalizada. Tasa de
157         aceptaci n final: {accept / n_iter:.3f}")
158     return np.array(samples)
159
160 # RESULTADOS
161
162 def summarize(samples, K):
163     burn_in = int(len(samples) * 0.3)
164     mean     = np.mean(samples[burn_in:], axis=0)
165     return mean[0:K], mean[K:2*K], mean[2*K:3*K]
166
167
168 if __name__ == "__main__":
169     samples = run_mcmc(x, y, K, n_iter=args.n_iter)

```

```

171     A_est, mu_est, sigma_est = summarize(samples, K)
172
173     print(f"\n--- RESUMEN ESTIMACIONES CADENA {args.chain_id} ---")
174     print("Amplitudes (A):      ", np.round(A_est, 4))
175     print("Medias (mu):          ", np.round(mu_est, 4))
176     print("Sigmas (sigma):       ", np.round(sigma_est, 4))
177
178     out = np.concatenate([A_est, mu_est, sigma_est])
179     np.savetxt(f"result_chain_{args.chain_id}.txt", out)
180     print(f" Resultados consolidados en: result_chain_{args.chain_id}
        }.txt\n")

```

6.3. analisis_cadenas.py

Listing 4: Análisis de resultados y generación de gráficas

```

1  import numpy as np
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  from scipy.signal import find_peaks
5
6  def leer_resultados_cadena(archivo, K):
7      with open(archivo, 'r') as f:
8          lines = f.readlines()
9          params = []
10         for line in lines[:K]:
11             A, mu, sigma = map(float, line.strip().split())
12             params.append([A, mu, sigma])
13         return np.array(params)
14
15  def modelo_gaussiano(x, params):
16      y = np.zeros_like(x)
17      for A, mu, sigma in params:
18          y += A * np.exp(-(x - mu)**2 / (2 * sigma**2))
19      return y
20
21  def main():
22      K = 3
23      resultados_cadenas = []
24
25      for chain_id in range(1, 5):
26          params = leer_resultados_cadena(f'result_chain_{chain_id}.
27              txt', K)
28          resultados_cadenas.append(params)
29
30      # Valores reales
31      valores_reales = np.array([

```

```

31         [10.0, 20.0, 2.0],
32         [15.0, 50.0, 3.0],
33         [12.0, 80.0, 1.5]
34     ])
35
36     # Promedio final
37     promedio_final = np.mean(resultados_cadenas, axis=0)
38
39     # Calcular errores
40     print("\n" + "="*70)
41     print("RESULTADOS FINALES - COMPARACION CON VALORES REALES")
42     print("="*70)
43     print(f"{'Par metro':<12} {'Real':<12} {'Estimado':<12} {'Error'
44           '(%)':<12}")
45     print("-"*70)
46
47     for i in range(K):
48         print(f"A{i+1}           {valores_reales[i,0]:<12.4f} {
49               promedio_final[i,0]:<12.4f} {abs((promedio_final[i,0]-
50               valores_reales[i,0])/valores_reales[i,0]*100):<12.2f}")
51         print(f"mu{i+1}           {valores_reales[i,1]:<12.4f} {
52               promedio_final[i,1]:<12.4f} {abs((promedio_final[i,1]-
53               valores_reales[i,1])/valores_reales[i,1]*100):<12.2f}")
54         print(f"sigma{i+1}       {valores_reales[i,2]:<12.4f} {
55               promedio_final[i,2]:<12.4f} {abs((promedio_final[i,2]-
56               valores_reales[i,2])/valores_reales[i,2]*100):<12.2f}")
57         print("-"*70)
58
59     # Graficar comparacion de cadenas
60     fig, axes = plt.subplots(3, 3, figsize=(15, 12))
61     parametros = ['Amplitud (A)', 'Centro ( )', 'Ancho ( )']
62
63     for i in range(K):
64         for j, param_idx in enumerate([0, 1, 2]):
65             ax = axes[i, j]
66             valores_cadenas = [resultados_cadenas[chain][i,
67                               param_idx] for chain in range(4)]
68             ax.bar(range(1, 5), valores_cadenas, alpha=0.7, label='
69                   Cadenas')
70             ax.axhline(y=valores_reales[i, param_idx], color='r',
71                       linestyle='--',
72                       label=f'Real: {valores_reales[i, param_idx]:.2
73                               f}')
74             ax.axhline(y=promedio_final[i, param_idx], color='g',
75                       linestyle='--',
76                       label=f'Promedio: {promedio_final[i, param_idx]
77                               :.2f}')

```

```

65         ax.set_xlabel('Cadena')
66         ax.set_ylabel(parametros[param_idx])
67         ax.set_title(f'Pico {i+1} - {parametros[param_idx]}')
68         ax.legend()
69         ax.grid(True, alpha=0.3)
70
71     plt.suptitle('Comparaci n de Cadenas MCMC por Par metro',
72                 fontsize=16)
73     plt.tight_layout()
74     plt.savefig(' analisis_cadenas.png', dpi=150)
75     print("\n Gr fica guardada: analisis_cadenas.png")
76
77     # Reconstruir se al
78     df = pd.read_csv('senal.csv')
79     x = df['x'].values
80     y_original = df['y'].values
81     y_reconstruida = modelo_gaussiano(x, promedio_final)
82
83     plt.figure(figsize=(12, 6))
84     plt.plot(x, y_original, 'b-', alpha=0.7, label='Se al Original
85              (con ruido)', linewidth=1)
86     plt.plot(x, y_reconstruida, 'r-', label='Se al Reconstruida',
87             linewidth=2)
88
89     # Marcar picos
90     for i in range(K):
91         plt.axvline(x=promedio_final[i, 1], color='gray', linestyle=
92                     ':', alpha=0.5)
93         plt.text(promedio_final[i, 1], promedio_final[i, 0],
94                  f'Pico {i+1}\n = {promedio_final[i,1]:.1f}',
95                  ha='center', va='bottom', fontsize=9)
96
97     plt.xlabel('x')
98     plt.ylabel('y')
99     plt.title('Se al Reconstruida vs Se al Original')
100    plt.legend()
101    plt.grid(True, alpha=0.3)
102    plt.savefig('senal_reconstruida.png', dpi=150)
103    print(" Gr fica guardada: senal_reconstruida.png")
104
105    plt.show()
106
107    if __name__ == "__main__":
108        main()

```

7. Conclusiones

En este se muestra exitosamente la implementación de un algoritmo MCMC paralelo para la inferencia bayesiana de parámetros con un modelo de suma gaussianas. Las principales observaciones son:

- El paralelismo a nivel de tareas permite una exploración más eficiente del espacio de parámetros sin aumentar significativamente el tiempo de cómputo.
- Aunque los errores de estimación en algunos parámetros fue alto el método ha demostrado que es una alternativa viable para este tipo de problemas. Una de las posibles causas de este error es el alto ruido de la señal o una configuración más estricta en la prior.
- El uso de múltiples cadenas con diferentes semillas aleatorias permite verificar la convergencia de cada pico gaussiano, al mostrar todas las cadenas la detección del mismo pico.
- La implementación es escalable y puede adaptarse fácilmente para un número diferente de picos gaussianos.